

객체지향언어1(7반)

과제 3 - 7장 연산자 중복과 9장 가상 함수

- 과제명: 실습문제 7장 6번, 9장 4번 풀기

- 학번: 2371172

- 이름: 안태호

- 트랙명: 빅데이터 트랙, 웹 공학 트랙

- 담당 교수명: 황기태

1. 실습문제 7장 6번 소스코드

Light.h

```
#ifndef LIGHT_H // LIGHT_H가 정의되어 있지 않다면

#define LIGHT_H // LIGHT_H를 정의함(중복 포함 방지)


#include <iostream> // 입출력 기능을 위한 표준 헤더 파일

using namespace std; // std 이름공간에 선언된 모든 이름에 std를 생략함


class Light { //Light 클래스 선언

    int size; // 빛의 크기를 저장하는 멤버 변수

public: // 아래 멤버들을 public으로 접근 지정

    Light(int size = 1); // 빛 크기를 초기화하는 생성자 선언

    Light& operator +=(Light& a); // 다른 Light 객체의 크기를 합치는 연산자 함수 선언

    Light& operator +=(int n); // 정수 값을 빛 크기에 더하는 연산자 함수 선언

    void operator ~(); // 빛 크기를 출력하는 연산자 함수 선언

    Light operator --(int x); // 후위 감소 연산자 함수 선언

    bool operator ==(int n); // 빛 크기와 정수 값을 비교하는 연산자 함수 선언

};


#endif //조건부 컴파일 종료
```

Light.cpp

```
#include "Light.h" // Light 클래스 선언 헤더 포함
```

```
//생성자 정의
```

```
Light::Light(int size) {  
    this->size = size; // 매개변수 size 값을 멤버 변수에 저장  
}
```

```
Light& Light::operator +=(Light& a) { // Light 객체끼리 더하는 연산자 함수 정의
```

```
    this->size += a.size; // 현재 객체 크기에 다른 객체 크기를 더함  
    a.size = 0; // 합쳐진 객체의 크기를 0으로 초기화  
    return *this; // 현재 객체를 반환
```

```
}
```

```
Light& Light::operator +=(int n) { // 정수를 더하는 연산자 함수 정의
```

```
    this->size += n; // 현재 객체 크기에 정수 값을 더함  
    return *this; // 현재 객체를 반환
```

```
}
```

```
void Light::operator ~() { // 빛 크기를 출력하는 연산자 함수 정의
```

```
    cout << "빛 크기 " << size << endl;
```

```
}
```

```
Light Light::operator --(int x) { // 후위 감소 연산자 함수 정의
```

```
    Light tmp = *this; // 현재 객체를 임시 객체에 저장  
    size--; // 현재 객체의 크기를 1 감소  
    return tmp; // 변경 전 객체를 반환
```

```
}
```

```
bool Light::operator ==(int n) { // 빛 크기와 정수를 비교하는 연산자 함수 정의
```

```
    if (size == n) // 빛 크기와 입력값이 같은 경우
```

```
        return true; // true 반환
```

```
    else // 빛 크기와 입력값이 다른 경우
```

```
        return false; // false 반환
```

```
}
```

main.cpp

```
#include "Light.h" // Light 클래스 선언 헤더 포함
```

```
#include <iostream> // 입출력 기능을 위한 표준 헤더 파일
```

```
using namespace std; // std 이름공간에 선언된 모든 이름에 std를 생략함
```

```
int main() {
```

```
    Light a(10), b(20), c, d(0); // 매개변수 생성자를 호출하여 Light 객체 생성
```

```
    (a += b) += 10; // a와 b를 합친 후 10을 추가
```

```
    ~a; // a 객체 정보 출력
```

```
    ~b; // b 객체 정보 출력
```

```
    ~c; // c 객체 정보 출력
```

```
    d = c--; // 후위 감소 연산 결과를 d에 저장
```

```
    ~d; // d 객체 정보 출력
```

```

if (c == 0) // c 객체의 크기가 0인지 비교

    cout << "빛 c는 꺼져 있습니다." << endl;

if (d == 0) // d 객체의 크기가 0인지 비교

    cout << "빛 d는 꺼져 있습니다." << endl;


return 0; // 프로그램을 종료하고 운영체제에 0을 반환

}

```

실행결과



The screenshot shows the Microsoft Visual Studio Debug Console window. The output text is as follows:

```

빛 크기 40
빛 크기 0
빛 크기 1
빛 크기 1
빛 c는 꺼져 있습니다.

```

Below the output, a message box is displayed with the following text:

```

C:\2026_class\객체지향언어\source\과제3\chap\x64\Debug\7-6.exe(프로세스 14832)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...

```

2. 실습문제 9장 4번 소스코드

AbstractPoint.h

```
#ifndef ABSTRACTPOINT_H // ABSTRACTPOINT_H가 정의되어 있지 않다면
#define ABSTRACTPOINT_H // ABSTRACTPOINT_H를 정의함(중복 포함 방지)

#include <iostream> // 입출력 기능을 위한 표준 헤더 파일
#include <string> // 문자열 사용을 위한 표준 헤더 파일

using namespace std; // std 이름공간에 선언된 모든 이름에 std를 생략함

class AbstractPoint { // AbstractPoint 추상 클래스 선언
protected: // 자식 클래스에서 접근 가능하도록 protected 지정
    string name; // 회원 이름을 저장하는 멤버 변수
    int point = 0; // 적립 포인트를 저장하는 멤버 변수
    int ratio; // 포인트 적립 비율을 저장하는 멤버 변수
public: // 아래 멤버들을 public으로 접근 지정
    // 회원 이름과 적립 비율을 초기화하는 생성자 선언
    AbstractPoint(string name, int ratio = 1);
    // 결제 금액에 따른 포인트 적립 함수 선언(순수 가상 함수)
    virtual void earn(int paymentAmount) = 0;
    virtual void show() = 0; // 포인트 정보를 출력하는 함수 선언(순수 가상 함수)
    int getPoint(); // 현재 포인트를 반환하는 함수 선언
};

#endif // 조건부 컴파일 종료
```

Platinum.h

```
#ifndef PLATINUM_H // PLATINUM_H가 정의되어 있지 않다면

#define PLATINUM_H // PLATINUM_H를 정의함(중복 포함 방지)


#include "AbstractPoint.h" // 부모 클래스 AbstractPoint 헤더 파일 포함


//AbstractPoint 클래스를 상속받는 Platinum 클래스 선언

class Platinum : public AbstractPoint {

private: // 아래 멤버들을 private으로 접근 지정

    int base; // 기본 적립 기준 금액을 저장하는 멤버 변수

    int bonusRatio; // 추가 적립 비율을 저장하는 멤버 변수

public: // 아래 멤버들을 public으로 접근 지정

    // 회원 정보와 적립 기준을 초기화하는 생성자 선언

    Platinum(string name, int ratio, int base, int bonusRatio);

    virtual void earn(int paymentAmount); // 결제 금액에 따른 포인트 적립 함수 선언

    virtual void show(); // 회원 포인트 정보를 출력하는 함수 선언

    void combine(Platinum& otherAccount); // 다른 계정의 포인트를 합치는 함수 선언

};


#endif // 조건부 컴파일 종료
```

AbstractPoint.cpp

```
#include "AbstractPoint.h" // AbstractPoint 클래스 선언 헤더 포함

// 생성자 정의
AbstractPoint::AbstractPoint(string name, int ratio) {

    this->name = name; // 매개변수 name 값을 멤버 변수에 저장
    this->ratio = ratio; // 매개변수 ratio 값을 멤버 변수에 저장
}

//현재 포인트를 반환하는 함수 정의
int AbstractPoint::getPoint() {

    return point; //현재 포인트 값을 반환
}
```

Platinum.cpp

```
#include "Platinum.h" // Platinum 클래스 선언 헤더 포함

// 생성자 정의
// 부모 클래스 생성자를 호출하여 name과 ratio 초기화
Platinum::Platinum(string name, int ratio, int base, int bonusRatio) : AbstractPoint(name, ratio) {

    this->base = base; // 매개변수 base 값을 멤버 변수에 저장
    this->bonusRatio = bonusRatio; // 매개변수 bonusRatio 값을 멤버 변수에 저장
}

// 포인트 적립 함수 정의
void Platinum::earn(int paymentAmount) {

    if (paymentAmount <= base) { // 결제 금액이 기준 금액 이하인 경우
```

```

        // 기본 적립 비율로 포인트를 계산하여 저장
        point += paymentAmount * ratio / 100;
    }

    else { //결제 금액이 기준 금액을 초과하는 경우
        // 추가 적립 비율로 포인트를 계산하여 저장
        point += paymentAmount * bonusRatio / 100;
    }
}

//회원 정보를 출력하는 함수 정의
void Platinum::show() {
    cout << name << "님*** 적립 포인트는 " << getPoint() << "점입니다." << endl;
}

// 다른 계정의 포인트를 합치는 함수 정의
void Platinum::combine(Platinum& otherAccount) {
    point += otherAccount.point; // 현재 계정에 다른 계정 포인트를 합산
    otherAccount.point = 0; // 다른 계정 포인트를 0으로 초기화
}

```

main.cpp

```

#include "Platinum.h" //Platinum 클래스 선언 헤더 포함

#include <iostream> //입출력 기능을 위한 표준 헤더 파일

using namespace std; // std 이름공간에 선언된 모든 이름에 std를 생략함

int main() {
    //매개변수 생성자를 호출하여 Platinum 객체 생성

```

```

Platinum hwang("황기태", 5, 100000, 7);

Platinum park("박채원", 5, 100000, 7);


hwang.earn(50000); // 황기태 회원에게 50000원 결제 포인트 적립

hwang.earn(200000); // 황기태 회원에게 200000원 결제 포인트 적립


park.earn(500000); // 박채원 회원에게 500000원 결제 포인트 적립


hwang.show(); // 황기태 회원 정보 출력

park.show(); // 박채원 회원 정보 출력


hwang.combine(park); // 박채원 회원 포인트를 황기태 회원에게 합산

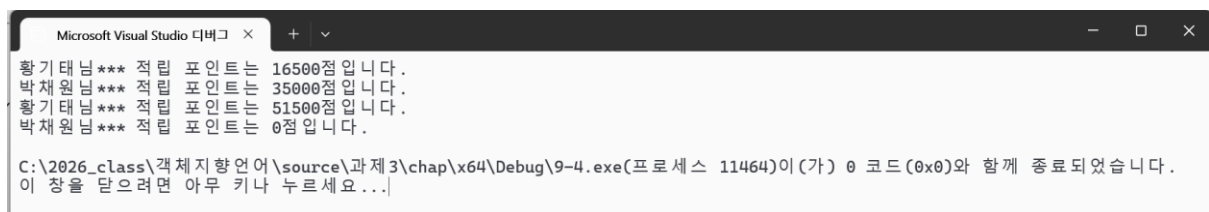

hwang.show(); // 합산 후 황기태 회원 정보 출력

park.show(); // 합산 후 박채원 회원 정보 출력


return 0; // 프로그램을 종료하고 운영체제에 0을 반환
}

```

실행결과



The screenshot shows the Microsoft Visual Studio Debug Console window. The title bar reads 'Microsoft Visual Studio 디버그' with standard window controls. The console output is as follows:

```

황기태님*** 적립 포인트는 16500점입니다.
박채원님*** 적립 포인트는 35000점입니다.
황기태님*** 적립 포인트는 51500점입니다.
박채원님*** 적립 포인트는 0점입니다.

C:\2026_class\객체지향언어\source\과제3\chap\x64\Debug\9-4.exe(프로세스 11464)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...

```

작성 소감

이번 과제를 작성하는 데 걸린 시간은 약 3시간 정도였습니다.

첫 번째 실습문제는 Light 클래스를 활용한 연산자 재정의 프로그램 만들기였습니다. 이번 문제에서는 생성자와 연산자 함수들을 클래스 내부에서 구현하고, 이를 헤더 파일과 구현 파일로 분리하여 작성해 보았습니다. 기존에는 연산자를 단순히 계산 기호처럼 사용했지만, 이번 과제를 통해 연산자도 클래스 객체에 맞게 새로운 기능을 정의할 수 있다는 점을 직접 경험할 수 있었습니다.

특히 += 연산자를 정의하는 과정이 인상 깊었습니다. Light 객체끼리 더하는 함수와 정수를 더하는 함수를 각각 구현하여 같은 연산자라도 매개변수 형태에 따라 다르게 동작하도록 만들었습니다. 또한 10장에서 학습한 템플릿 함수 개념을 떠올리며, 자료형이나 객체 종류가 달라지더라도 공통된 방식으로 처리할 수 있도록 일반화하는 방법도 적용해 볼 수 있겠다는 생각이 들었습니다.

Light 객체를 매개변수로 받을 경우에는 다른 객체의 size 값을 현재 객체에 더하고, 더해진 객체의 size를 0으로 초기화하도록 구현했습니다. 또한 함수 반환형을 객체 참조형으로 선언하고 return *this;를 사용하여 (a += b) += 10과 같은 연쇄 호출이 가능하도록 구현했습니다.

또한 후위 감소 연산자인 operator--(int x)를 구현하면서 후위 연산의 동작 방식도 다시 확인할 수 있었습니다. 처음에는 단순히 size 값을 감소시키면 된다고 생각했지만, 후위 연산은 감소 이전의 값을 반환해야 하기 때문에 임시 객체를 생성하여 현재 상태를 저장한 후 size를 감소시키는 과정을 구현했습니다.

두 번째 실습문제는 Platinum 회원의 포인트를 관리하는 프로그램 만들기였습니다. 9-4 문제에서는 추상 클래스와 상속을 사용하여 객체 지향 프로그래밍의 구조를 직접 구현해 볼 수 있었습니다. 부모 클래스인 AbstractPoint에는 회원 이름, 포인트, 적립 비율과 같은 공통 정보를 저장하고, 포인트 적립 함수 earn()과 출력 함수 show()를 순수 가상 함수로 선언하였습니다. 이후 자식 클래스인 Platinum에서 이를 상속받아 실제 기능을 구현하였습니다.

프로그램의 실행 과정을 살펴보면 먼저 main 함수에서 hwang 객체와 park 객체를 생성하는 순간 Platinum 생성자가 호출됩니다. 이때 생성자 초기화 리스트를 통해 부모 클래스인 AbstractPoint 생성자가 먼저 호출되어 name과 ratio 값이 초기화되고, 이후 Platinum 클래스 내부의 base와 bonusRatio가 초기화됩니다.

그 후 `hwang.earn(50000)`을 호출하면 결제 금액이 기준 금액보다 작기 때문에 기본 적립 비율인 `ratio`를 이용하여 포인트가 계산됩니다. 반대로 `hwang.earn(200000)`이나 `park.earn(500000)`을 호출할 경우에는 기준 금액을 초과하므로 `bonusRatio`를 적용하여 추가 적립 포인트가 계산됩니다. 이후 `show()` 함수를 호출하면 회원 이름과 적립 포인트가 출력되며, `hwang.combine(park)`를 호출하면 `park` 객체가 가지고 있던 포인트가 `hwang` 객체로 옮겨지고 `park`의 포인트는 0으로 초기화됩니다. 이 과정에서는 매개변수를 참조형으로 전달하여 실제 객체 자체를 수정할 수 있게 구현했습니다. 참조 매개변수를 사용했기 때문에 함수 내부에서 `otherAccount.point` 값을 변경하면 복사본이 아닌 원본 객체인 `park`의 포인트가 직접 변경되어 0으로 초기화될 수 있었습니다.

특히 이번 과제에서는 동적 바인딩의 개념이 매우 중요하다고 느꼈습니다. `AbstractPoint` 클래스에서 `earn()` 함수와 `show()` 함수를 순수 가상 함수로 선언하고 `virtual` 키워드를 사용하면 부모 클래스 포인터가 자식 객체를 가리키는 상황에서 실행 시점에 객체 타입을 확인하여 자식 클래스의 함수가 호출된다는 점도 함께 이해할 수 있었습니다.. 이것이 바로 동적 바인딩이며, 실행 중에 어떤 함수가 호출될지를 결정하는 객체 지향 프로그래밍의 중요한 특징이라는 점을 이해할 수 있었습니다.

또한 이번 과제는 단순히 상속 문법을 사용하는데 있는 것이 아니라 공통 기능과 세세한 기능을 분리하여 설계하는 과정이 중요하다고 생각했습니다. 부모 클래스에는 공통 기능을 작성하고, 자식 클래스에서는 필요한 기능만 재정의함으로써 코드 재사용성과 확장성이 높아진다는 것을 알 수 있었습니다.

이번 과제를 수행하며 느낀 소감은 추상 클래스, 상속, 순수 가상 함수, 동적 바인딩 같은 개념들은 다소 어렵게 느껴졌지만, 실제 프로그램 안에서 직접 구현하고 실행해 보니까 각 개념의 역할과 필요성을 더욱 명확하게 이해할 수 있었습니다. 단순히 기능을 구현하는 것이 아니라 부모 클래스와 자식 클래스의 역할을 구분하고, 공통 기능과 세부 기능을 나누어 설계하는 과정이 객체 지향 프로그래밍의 핵심이라는 점도 느낄 수 있었습니다.

또한 프로그램의 실행 순서를 하나씩 따라가며 객체 생성 과정, 함수 호출 과정, 포인트 적립 과정 등을 분석해 보면서 코드가 어떤 방식으로 동작하는지 이전보다 더 깊이 생각하게 되었습니다. 특히 `virtual` 키워드를 사용했을 때 실행 시점에 실제 객체의 함수가 호출되는 동적 바인딩 과정을 직접 확인하며 객체 지향 프로그래밍은 프로그램의 유연성과 확장성을 높여주는 중요한 설계 방식이라는 점을 체감할 수 있었습니다.

이번 과제를 통해 객체 지향 프로그래밍의 개념을 실제 코드로 적용하며 그 중요성과 활용성을 직접 느낄 수 있었던 유익한 시간이었습니다.